

ШАБЛОННЫЕ ФУНКЦИИ

Семинар 14

Шаблонная функция (обобщенная функция, template function)

- определяет общий набор операций, которые предназначены для применения к данным различных типов;
- шаблонная функция перегружает сама себя.

```
template <typename T> тип имя_функции(список_параметров)
{
    ...
}
```

Шаблонные функции

```
1  #include <iostream>
2
3  template <class T> const T& _max(const T& x1, const T& x2)
4  {
5      return (x1 > x2) ? x1 : x2;
6  }
7
8  int main(int argc, char **argv)
9  {
10     int i = _max(3, -6);
11     std::cout << i << "\n";
12
13     double d = _max(5.12, -3.14);
14     std::cout << d << "\n";
15
16     std::string str = _max(std::string("Mirea"), std::string("MIREA"));
17     std::cout << str << "\n";
18     return 0;
19 }
```

- ключевое слово `template` сообщает компилятору, что дальше объявляются параметры шаблона;
- ключевые слова `typename` и `class` используются для создания типов параметров шаблона. В базовом случае использования (как в примере) разницы между ними нет.

Вывод типов

- как правило, при инстанцировании шаблонной функции компилятор может сам вывести шаблонный тип исходя из типов переданных данной функции параметров (type deduction);
- в некоторых случаях компилятор не способен самостоятельно вывести тип, например:
 - у шаблонной функции нет параметров,
 - параметрам одного и того же шаблонного типа переданы значения разных типов (например, `int` и `long`),
 - необходимо «навязать» параметрам преобразование типов;
- в таком случае при вызове шаблонной функции необходимо задать тип в явном виде в треугольных скобках после имени функции.

Шаблонные функции

```
1  #include <iostream>
2
3  template <class T> const T& _max(const T& x1, const T& x2)
4  {
5      return (x1 > x2) ? x1 : x2;
6  }
7
8  int main(int argc, char **argv)
9  {
10     int i = _max(3, -6);
11     std::cout << i << "\n";
12
13     double d = _max(5.12, -3.14);
14     std::cout << d << "\n";
15
16     std::string str = _max(std::string("Mirea"), std::string("MIREA"));
17     std::cout << str << "\n";
18     return 0;
19 }
```

Шаблонные функции

1. При вызове шаблонной функции, компилятор создает версию этой функции для конкретного типа, т.е. создается ее специализация (конкретизация, порожденная функция (generated function)).
2. Действие порождения функции определяют как ее реализацию (instantiating).
3. Сама по себе шаблонная функция не является ни функцией, ни типом. Если не была создана хотя бы одна специализация (т.е. никак не были определены шаблонные типы), она и вовсе не компилируется, т.е. никак не отображается из исходного кода в объектный код.

Явная специализация

Если необходимо определить отдельную реализацию шаблонной функции для некоторых типов, можно переопределить такую функцию вручную. Это явная специализация шаблона (implicit specialization).

```
3  ✓ template <class T> const T& _max(const T& x1, const T& x2)
4      {
5          return (x1 > x2) ? x1 : x2;
6      }
7
8      template<>
9  ✓ const int& _max(const int& x1, const int& x2)
10     {
11         return x1 > x2 ? x1 : x2;
12     }
```

Нетипизированные параметры

В шаблоне можно
определить параметры
конкретных типов:

```
1  #include <iostream>
2
3  // печать значения val N раз
4  template <typename T, unsigned N = 1>
5  void print(const T& val)
6  {
7      for (unsigned i = 0; i < N; ++i)
8          std::cout << val << " ";
9      std::cout << "\n";
10 }
11
12 int main(int argc, char** argv)
13 {
14     print<double, 5>(3.14);
15     return 0;
16 }
```

Нетипизированные параметры

```
1  #include <iostream>
2
3  template <typename T, size_t N> T average(const T (&)[N]);
4
5  int main()
6  {
7      int numbers1[]{1, 2, 3, 4, 5};
8      std::cout << average(numbers1) << std::endl;    // 3
9
10     double numbers2[]{1.1, 3.2, 4.3, 5.4, 6.5, 2.6};
11     std::cout << average(numbers2) << std::endl;    // 3.85
12 }
13
14 template <typename T, size_t N> T average(const T (&data)[N])
15 {
16     T result {};
17     for(unsigned i{}; i < N; i++)
18     {
19         result += data[i];
20     }
21     return result / N;
22 }
```

Задачи

1. Реализовать шаблонную функцию, реализующую возведение в степень. Объявить явную специализацию для строкового типа (дублировать строку).

2. Доработать класс для работы с ini-файлами:
 - перевести на шаблоны функции чтения;
 - перевести на шаблоны функции записи;
 - сделать явную специализацию для необходимых типов.